Design Document

Command-line Personal Information Management System

COMP3211 Software Engineering, Fall 2026

Group 44

November 11, 2026

Contents

1 Architecture Design **2**
- 1.1 Scope and Architectural Pattern . . . 2
- 1.2 MVC Responsibilities and Dependencies . . . 2
- 1.3 System Boundaries and Shared Conventions . . . 4
- 1.4 Key Design Decisions . . . 4

2 Components and Interface Contracts **4**
- 2.1 Packages, Notation and Exceptions . . . 4
- 2.2 Record Hierarchy and Invariants . . . 7
- 2.3 Construction, Collection and Snapshots . . . 7
- 2.4 Search Conditions and Parsing . . . 9
- 2.5 Persistence and File Format . . . 11
- 2.6 Controller and Console Interfaces . . . 15
- 2.7 Failure and State Contracts . . . 17

3 Example: Search and Update an Event **18**
- 3.1 User Goal and Demonstration . . . 18
- 3.2 Object Collaboration . . . 18
- 3.3 Rejected Update and Preserved State . . . 20
- 3.4 Save and Load Round Trip . . . 21

4 Design Evaluation **21**
- 4.1 Efficiency and Resource Use . . . 21
- 4.2 Modularity, Extensibility and Trade-offs . . . 22
- 4.3 User-story-to-design Mapping . . . 22
- 4.4 Implementation Checks . . . 22
- 4.5 Functional Test Cases and Expected Outcomes . . . 23

1 Architecture Design

1.1 Scope and Architectural Pattern

PIM is a local command-line application for one user. It targets Java 17 and uses the Java standard library. Users can create notes, tasks, events and contacts, then update, search, display or delete them. Save and load commands keep the collection in a `.pim` file. These operations cover the eleven user stories in Appendix B of the course brief.

We use Model–View–Controller (MVC) with a passive console view. PIM has record rules that apply to every command, such as required fields and the order of event times. Keeping these rules in the model lets create, update and load use the same validation. Unit tests can call the model directly without entering commands or reading terminal output.

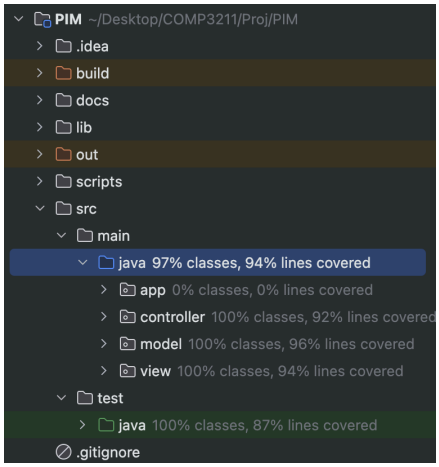
The controller handles the session. It reads a command, calls the required model operation and asks the view to show the result. It also tracks unsaved changes and asks for confirmation before data is discarded. These decisions depend on the user's session, so they belong in `PimController`.

`ConsoleView` reads input and formats output. Changing a prompt or the way record fields are printed does not change validation or search rules. The model imports neither the controller nor the view. This separation gives each package a clear role in the CLI and keeps the model tests independent of terminal interaction.

1.2 MVC Responsibilities and Dependencies

- **Model:** `RecordBook` owns the active collection and identifier counter. Immutable PIR classes and `RecordFactory` enforce field rules. Criteria and their parser define queries. `RecordCodec` and `PimFileStorage` implement file representation and input/output. All reside in `model` or its subpackages and import neither controller nor view.
- **Controller:** `PimController` runs the session, invokes `CommandParser`, calls model services and requests output. It owns `running` and `dirty`, catches recoverable application errors, and asks for confirmation before overwriting a file, discarding unsaved data on load, or exiting with unsaved changes.
- **View:** `ConsoleView` reads command lines and confirmations. It displays the welcome message, command help, complete records and errors. It has no record-book reference and cannot mutate records or interpret search expressions.
- **Entry point:** `Main` constructs the collaborators, injects them into the controller and calls `run`. The controller explicitly requests display updates; no model-to-view notifications are required.

For each command, the controller parses the input and calls the model. It then asks the view to display the result or error. The view reads record fields through PIR accessors. Save and load use snapshots to pass a complete collection between the controller and storage service.



Project structure. IntelliJ IDEA shows separate `app`, `controller`, `model` and `view` packages under `src/main/java`. Model classes are grouped in `model` and its subpackages.

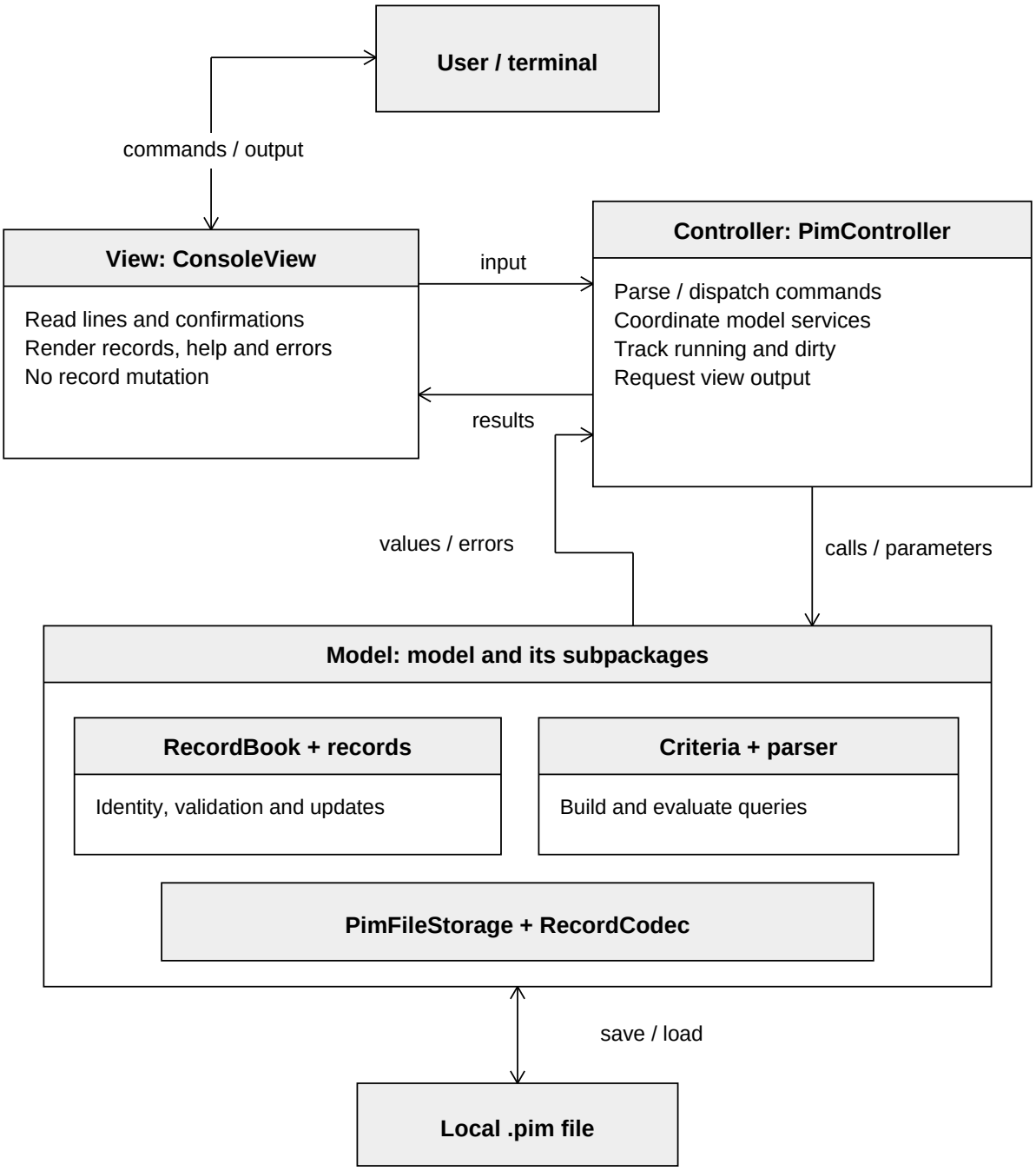


Figure 1: Controller-driven MVC. The model returns values and errors to the controller; the passive view receives presentation requests from that controller.

1.3 System Boundaries and Shared Conventions

Commands run sequentially against **one active collection**. Edits remain in memory until the user saves. Loading replaces the collection only after the complete file passes validation. The application runs locally with file storage and a terminal interface. Network services, database storage, concurrent editing and a GUI are outside scope.

Time fields use `LocalDateTime`, local time and minute precision. Input, display and saved values use `yyyy-MM-dd HH:mm`. An event's alarm is a required time field that users can update, search, display and save. As confirmed with the instructor, reminders and notifications are not required. Text searches are case-sensitive. The command help uses the same rules.

1.4 Key Design Decisions

The table gives the reasons and costs of the main design choices. The following sections describe how each component works.

Choice	Reason	Cost
D1 Immutable PIRs; validated replacement	Validate cross-field rules before changing the collection; identity remains stable.	Allocate a replacement object, even for an accepted equal-value update.
D2 Enums and typed optional field access	Separate field categories and expose absence without reflection or unsafe casts.	Maintain mappings between command names, enums and record fields.
D3 Parse once into a Criterion tree	Separate syntax errors from evaluation; compose Boolean conditions and test predicates independently.	Maintain a tokenizer, precedence rules and condition classes.
D4 Explicit save; fully validated load	Give users control over persistence and install only a complete valid collection.	Track unsaved state, request confirmation and retain a candidate collection during loading.

2 Components and Interface Contracts

2.1 Packages, Notation and Exceptions

Package	Components
`app`	`Main`
`model`	`RecordBook`, `RecordFactory`, `RecordSnapshot`
`model.records`	`PIR`, `Note`, `Task`, `Event`, `Contact`, `RecordType`, `FieldName`, `RecordTime`
`model.criteria`	`Criterion`, six condition classes, `CriterionParser`, `TimeOperator`, and the package-private `CriterionLexer`
`model.persistence`	`RecordCodec`, `PimFileStorage`
`model.exceptions`	`PimException` and its four model subclasses
`controller`	`PimController`, `CommandParser`, `Command`, `CommandType`, `CommandParseException`
`view`	`ConsoleView`

Methods are public unless marked protected. Fields are private and final unless marked mutable. Signatures include parameter types, return types and checked exceptions. Returned collections are read-only. Null record fields fail validation; null service dependencies may raise `NullPointerException`.

In diagrams, triangles mark supertypes, dashed arrows show dependencies, and solid associations show stored references. Numbers give multiplicities.

`PimException` extends `Exception`. Its subclasses below report recoverable errors. All provide `(String message)` and `(String message, Throwable cause)` constructors.

Subclass of PimException	Meaning and package
`ValidationException`	Invalid record, criterion parameters or snapshot; `model.exceptions`.
`RecordNotFoundException`	Unknown or non-positive lookup identifier; `model.exceptions`.
`CriterionParseException`	Invalid search expression; `model.exceptions`.
`StorageException`	File access, decoding or file-schema failure; `model.exceptions`.
`CommandParseException`	Invalid command syntax; `controller`.

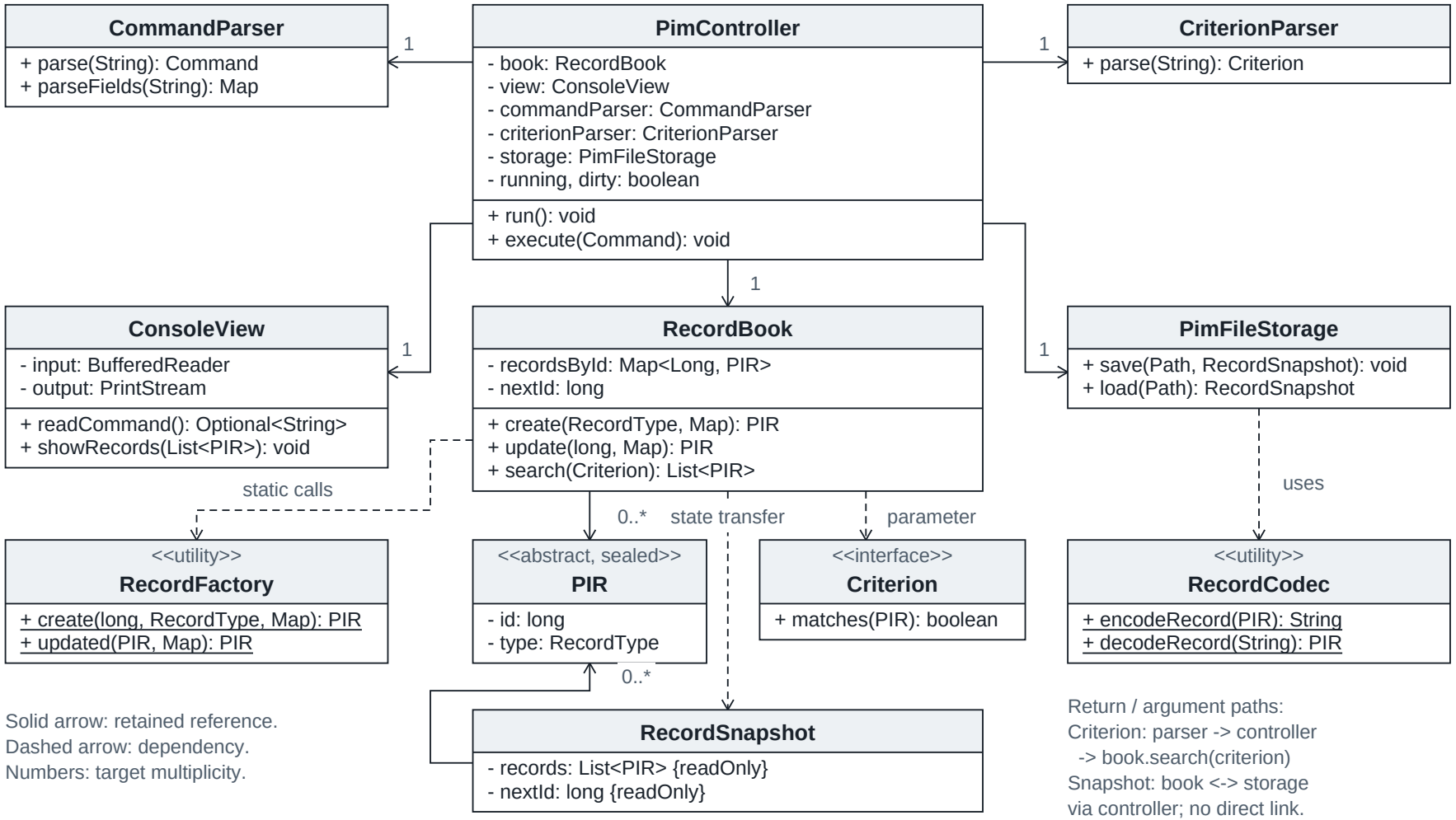


Figure 2: System overview. Solid associations identify retained references; dashed dependencies identify service or static utility use. Criterion and snapshot values cross operation boundaries without becoming retained book fields. The lower paths distinguish inputs, returned values and the point of record mutation.

2.2 Record Hierarchy and Invariants

`RecordType` contains `NOTE`, `TASK`, `EVENT`, `CONTACT`. `FieldName` contains `TEXT`, `DESCRIPTION`, `DEADLINE`, `START`, `ALARM`, `NAME`, `ADDRESS`, `MOBILE`. Parsers map lower-case user names to these enums.

`PIR` is an abstract sealed class with four final subclasses. It holds `long id` and `RecordType type`. IDs are positive and stay the same after updates, saves and loads. A typed accessor returns an empty optional if the field does not apply to the record. Getters and accessors declare no checked exceptions.

```
protected PIR(long id, RecordType type) throws ValidationException
long getId()
RecordType getType()
Optional<String> getTextField(FieldName field)
Optional<LocalDateTime> getTimeField(FieldName field)
```

Subclass	Additional fields	Public getters
`Note`	`String text`	`getText()`
`Task`	`String description`; `LocalDateTime deadline`	`getDescription()`, `getDeadline()`
`Event`	`String description`; `LocalDateTime start`, `alarm`	`getDescription()`, `getStart()`, `getAlarm()`
`Contact`	`String name`, `address`, `mobile`	`getName()`, `getAddress()`, `getMobile()`

Each getter returns its field's declared type. All subclasses override the text accessor; Task and Event also override the time accessor, using the base signatures. Constructors have the following contracts:

```
Note(long id, String text) throws ValidationException
Task(long id, String description, LocalDateTime deadline)
    throws ValidationException
Event(long id, String description, LocalDateTime start,
      LocalDateTime alarm) throws ValidationException
Contact(long id, String name, String address, String mobile)
    throws ValidationException
```

Text must be non-null, non-blank and valid Unicode. The model rejects unpaired UTF-16 surrogates and keeps valid text as entered. Mobile numbers are strings, so leading zeroes and plus signs are preserved. `RecordTime` accepts years `0000` to `9999` and requires seconds and nanoseconds to be zero. Times cannot be null. An event's alarm must be at or before its start; past times are allowed. Records have no public setters, so these rules cannot be bypassed after construction.

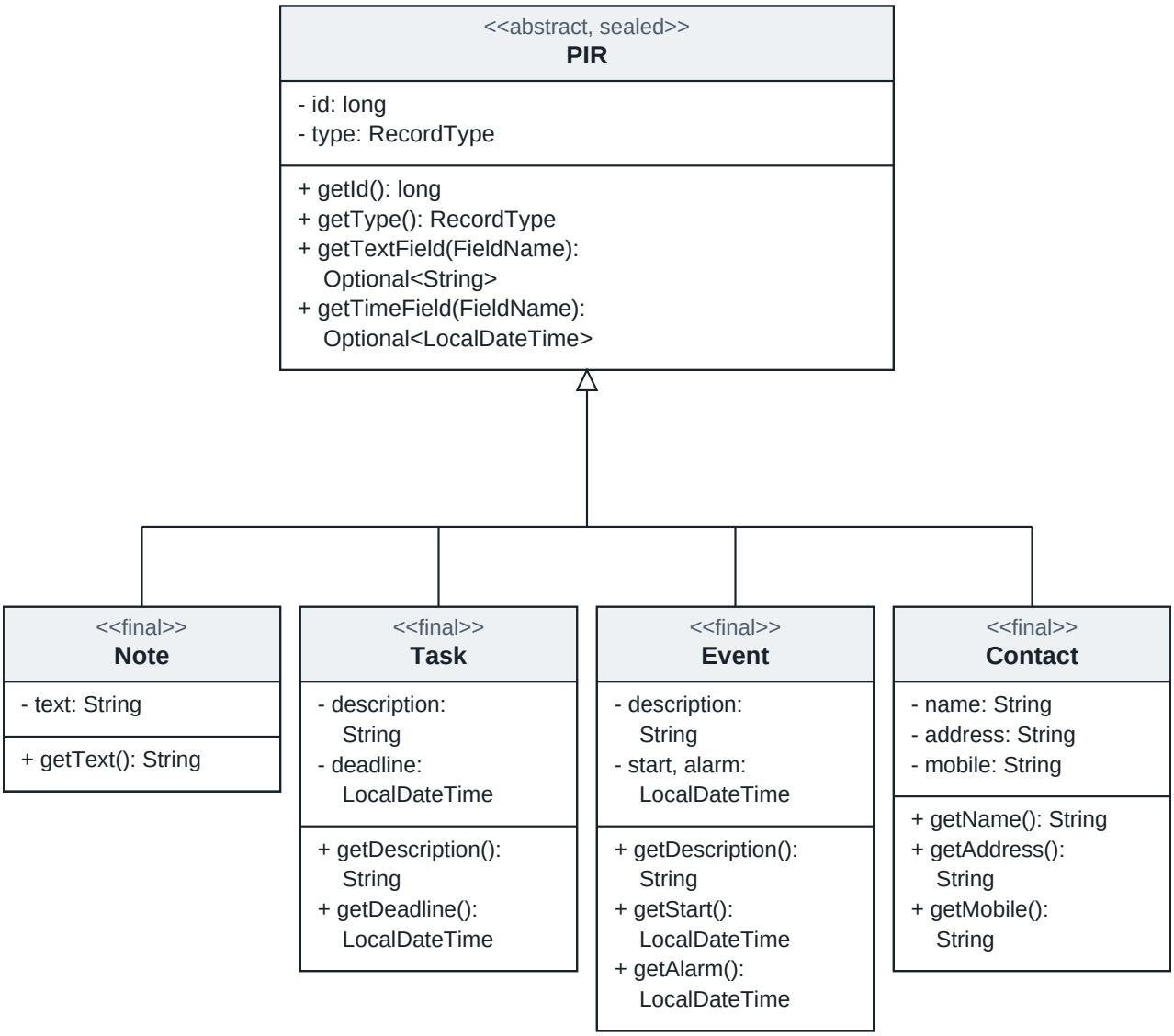
2.3 Construction, Collection and Snapshots

2.3.1 RecordFactory

This stateless utility has no public constructor. It converts field strings and constructs complete validated records, including time parsing.

```
static PIR create(long id, RecordType type,
                  Map<FieldName, String> fields)
    throws ValidationException
static PIR updated(PIR original, Map<FieldName, String> changes)
    throws ValidationException
```

`create` requires all fields for the selected type and rejects extra fields. `updated` requires at least one field assignment; the supplied value may equal the current value. It combines the supplied values with the existing fields and returns a new record with the same ID and type. Missing, extra or invalid values raise `ValidationException`. The original record stays unchanged.



All fields are final. Constructors validate full records; field-access overrides follow the base contract. Event invariant: alarm <= start.

Figure 3: Sealed PIR hierarchy. Each final subclass inherits the ID and type, provides access to its fields, and validates values in its constructor.

2.3.2 RecordSnapshot

The immutable snapshot contains `List<PIR> records` and `long nextId`. Its constructor copies the list and checks each record. Records cannot be null, and their IDs must be positive and unique. The next ID must be positive and greater than every stored ID. Invalid snapshots raise `ValidationException`; the getters declare no checked exceptions.

```
RecordSnapshot(List<PIR> records, long nextId)
    throws ValidationException
List<PIR> getRecords()
long getNextId()
```

2.3.3 RecordBook

The book has mutable `Map<Long, PIR> recordsById` and `long nextId`. Its no-argument constructor creates an empty collection with counter 1 and declares no checked exceptions.

```
PIR create(RecordType type, Map<FieldName, String> fields)
    throws ValidationException
PIR get(long id) throws RecordNotFoundException
List<PIR> getAll()
PIR update(long id, Map<FieldName, String> changes)
    throws RecordNotFoundException, ValidationException
void delete(long id) throws RecordNotFoundException
List<PIR> search(Criterion criterion)
RecordSnapshot snapshot() throws ValidationException
void replaceState(RecordSnapshot snapshot)
```

Creation checks the new record before storing it or advancing the ID counter. It fails if no more IDs are available. Update asks the factory to validate a replacement before changing the map. Deleted IDs are not reused. A rejected creation or update leaves the records, next ID and unsaved state unchanged.

`getAll` and `search` return records in ascending ID order. Search takes a valid, non-null criterion and leaves the collection unchanged.

`snapshot` builds a validated snapshot containing the current records and next ID. `replaceState` builds a map from that snapshot, then replaces the current map and counter. The same `RecordBook` object remains in use. `getAll`, `search` and `replaceState` declare no checked exceptions. Snapshots and returned lists can refer to the same PIR objects because those objects are immutable.

2.4 Search Conditions and Parsing

2.4.1 Criterion Interface and Implementations

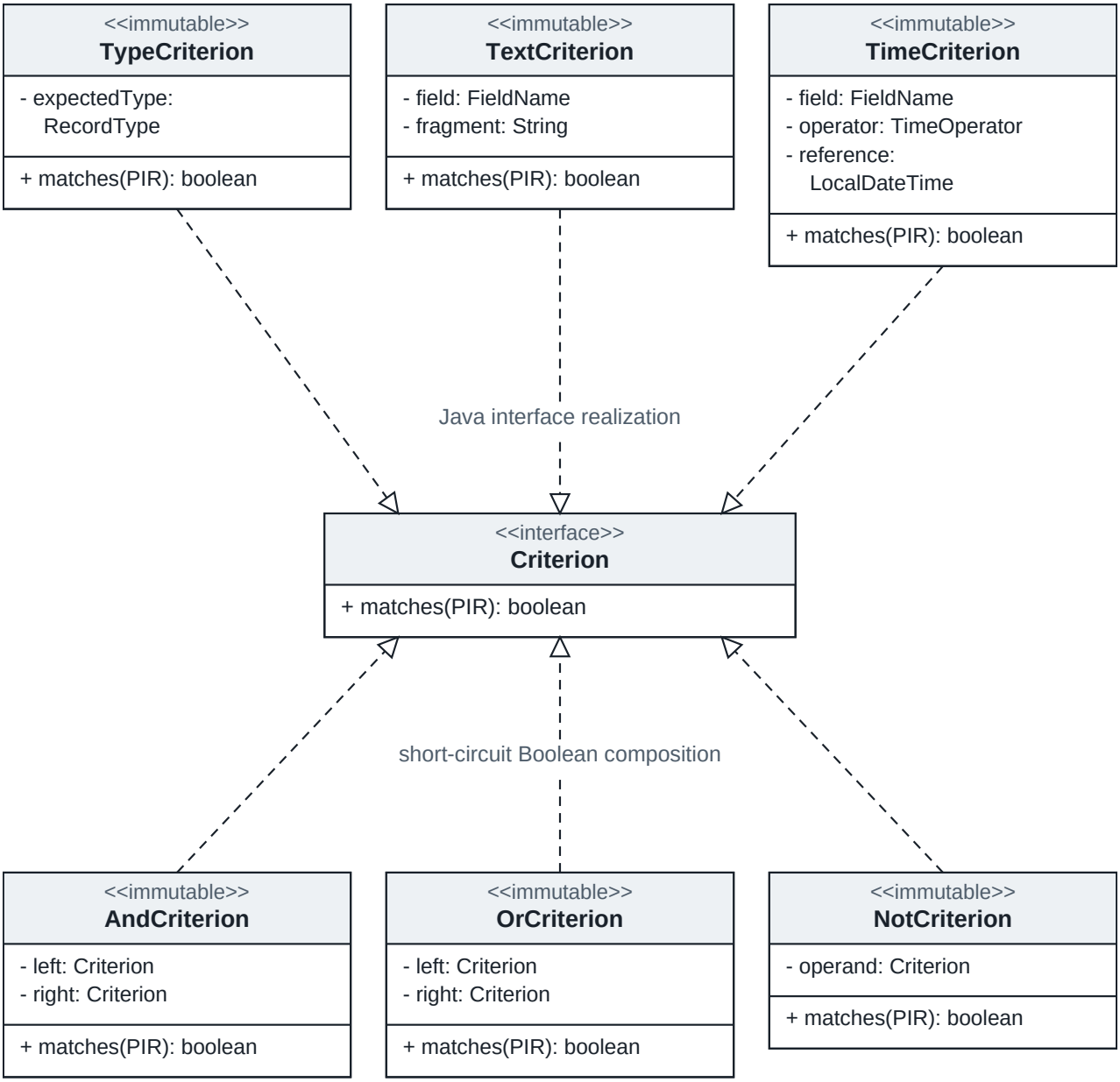
`Criterion` declares `boolean matches(PIR record)` for a non-null valid PIR. Its immutable implementations evaluate without checked exceptions or side effects. Constructors below validate their parameters and declare `ValidationException`.

```
TypeCriterion(RecordType expectedType) throws ValidationException
TextCriterion(FieldName field, String fragment)
    throws ValidationException
TimeCriterion(FieldName field, TimeOperator operator,
              LocalDateTime reference) throws ValidationException
AndCriterion(Criterion left, Criterion right)
    throws ValidationException
OrCriterion(Criterion left, Criterion right)
    throws ValidationException
NotCriterion(Criterion operand) throws ValidationException
```

Constructor parameters are stored in fields with the same names and types. `TypeCriterion` compares the record type. `TextCriterion` checks whether a text field contains a non-empty substring, with case-sensitive matching. `TimeCriterion` compares a time field with a reference time at minute

precision. Its `BEFORE`, `AFTER` and `EQUAL` operators mean strict less-than, strict greater-than and equality.

AND and OR each hold two non-null criteria and use short-circuit evaluation. NOT holds one criterion and reverses its result. Different criteria can use the same operand object. A comparison against a missing field is false; NOT can turn that result into true. A type condition restricts such a query to the intended record type.



Each binary operand reference has multiplicity 1; NOT has one operand. All conditions share the same side-effect-free matches contract.

Figure 4: Criterion implementations and their operands. Each class implements matches; AND, OR and NOT combine criteria into larger expressions.

2.4.2 CriterionParser and Search Grammar

The parser has a no-argument constructor, no persistent mutable state, and one public operation:

```
Criterion parse(String expression) throws CriterionParseException
```

For each expression, `CriterionParser` creates a package-private `CriterionLexer`. The lexer keeps quoted values intact and records token positions. The parser uses recursive descent to build the condition tree and checks that no input is left over. Invalid fields, types, operators, times or expression syntax cause `CriterionParseException`, with a position and reason. Validation errors from criterion constructors are converted to the same exception.

```
expression   := orExpr
orExpr       := andExpr { "||" andExpr }
andExpr      := unaryExpr { "&&" unaryExpr }
unaryExpr    := "!" unaryExpr | "(" expression ")" | predicate
predicate    := "type" "=" recordType
              | textField "contains" quotedString
              | timeField timeOperator quotedTime
timeOperator := "<" | ">" | "="
recordType   := "note" | "task" | "event" | "contact"
```

Parentheses set the grouping. Within each group, NOT (`!`) binds first, then AND (`&&`), then OR (`||`). Repeated binary operators associate left. Text fields are text, description, name, address and mobile. Time fields are deadline, start and alarm. Literals use double quotes and support escaped quotes and backslashes. Times follow Section 1. The parser accepts this search syntax and does not execute Java or scripts.

An expression can contain up to `MAX_EXPRESSION_LENGTH = 4096` UTF-16 code units. The parser allows `MAX_DEPTH = 32` levels of negation or grouping, and the lexer allows `MAX_TOKENS = 256` input tokens. Exceeding a limit raises `CriterionParseException`.

2.5 Persistence and File Format

2.5.1 RecordCodec

This stateless utility has no public constructor. It encodes and decodes one record using the schema in Section 2.5.3. Decoding calls the factory to construct and validate the record.

```
static String encodeRecord(PIR record)
static PIR decodeRecord(String line) throws ValidationException
```

Encoding returns one record line and declares no checked exceptions. Decoding raises `ValidationException` for malformed columns or invalid record values.

2.5.2 PimFileStorage

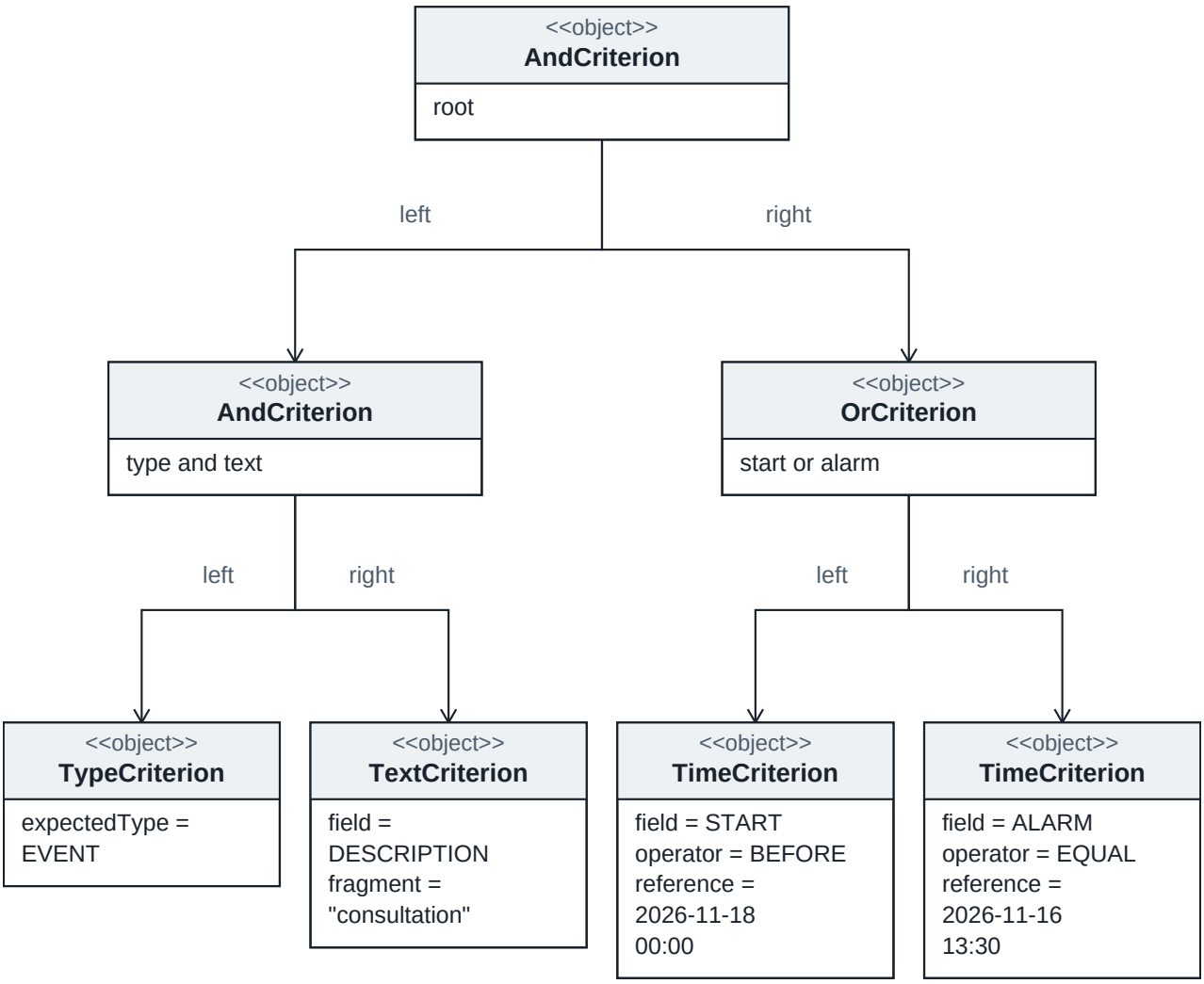
The stateless service has a no-argument constructor without checked exceptions.

```
void save(Path path, RecordSnapshot snapshot) throws StorageException
RecordSnapshot load(Path path) throws StorageException
```

Both methods accept only filenames ending in lower-case `.pim`. A different extension raises `StorageException`, even when called directly without the controller. The controller asks for confirmation. The storage service validates files and handles I/O.

Loading requires a regular file and follows a symbolic link to one. An existing save destination must itself be a regular file. Saving rejects symbolic links and directories.

Save writes the complete snapshot to a temporary file in the destination directory and closes it. It then replaces the target using `Files.move` with `ATOMIC_MOVE`. If the filesystem raises `AtomicMoveNotSupportedException`, save falls back to a normal replacement. Other failures are reported, and the temporary file is removed where possible. A failed save preserves the in-memory records, next ID and unsaved state. Failed temporary-file writes leave the target untouched, but a failed non-atomic replacement may affect the old file. Power-loss recovery and cross-process locking are outside scope.



Object graph for the event-search example. Every operand is one Criterion reference; left-associative AND and the parenthesised OR are preserved.

Figure 5: Condition tree for the consultation-event example in Section 3. Two outer AND nodes enforce type and description; the parentheses contain an OR between the start and alarm comparisons.

Load reads a candidate collection and checks every record, the count and the next ID before returning a snapshot. File-format or validation errors raise `StorageException`, with the cause and line number where available. The controller calls `replaceState` only after a successful load. A failed load leaves the current records, next ID and unsaved state unchanged.

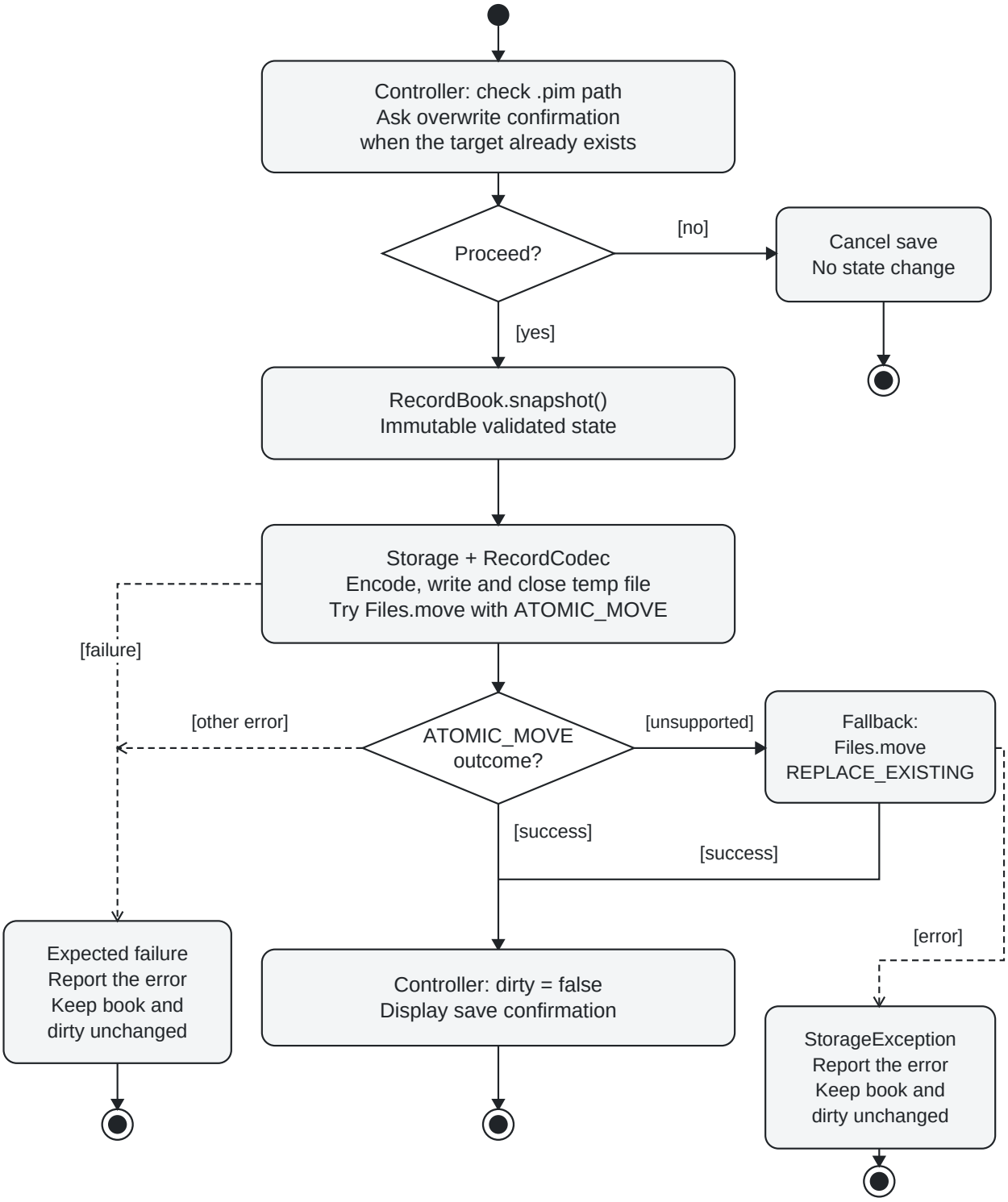


Figure 6: Save collaboration. Overwrite confirmation belongs to the controller; snapshot/file validation belongs to the model. Only successful destination replacement clears the session dirty flag.

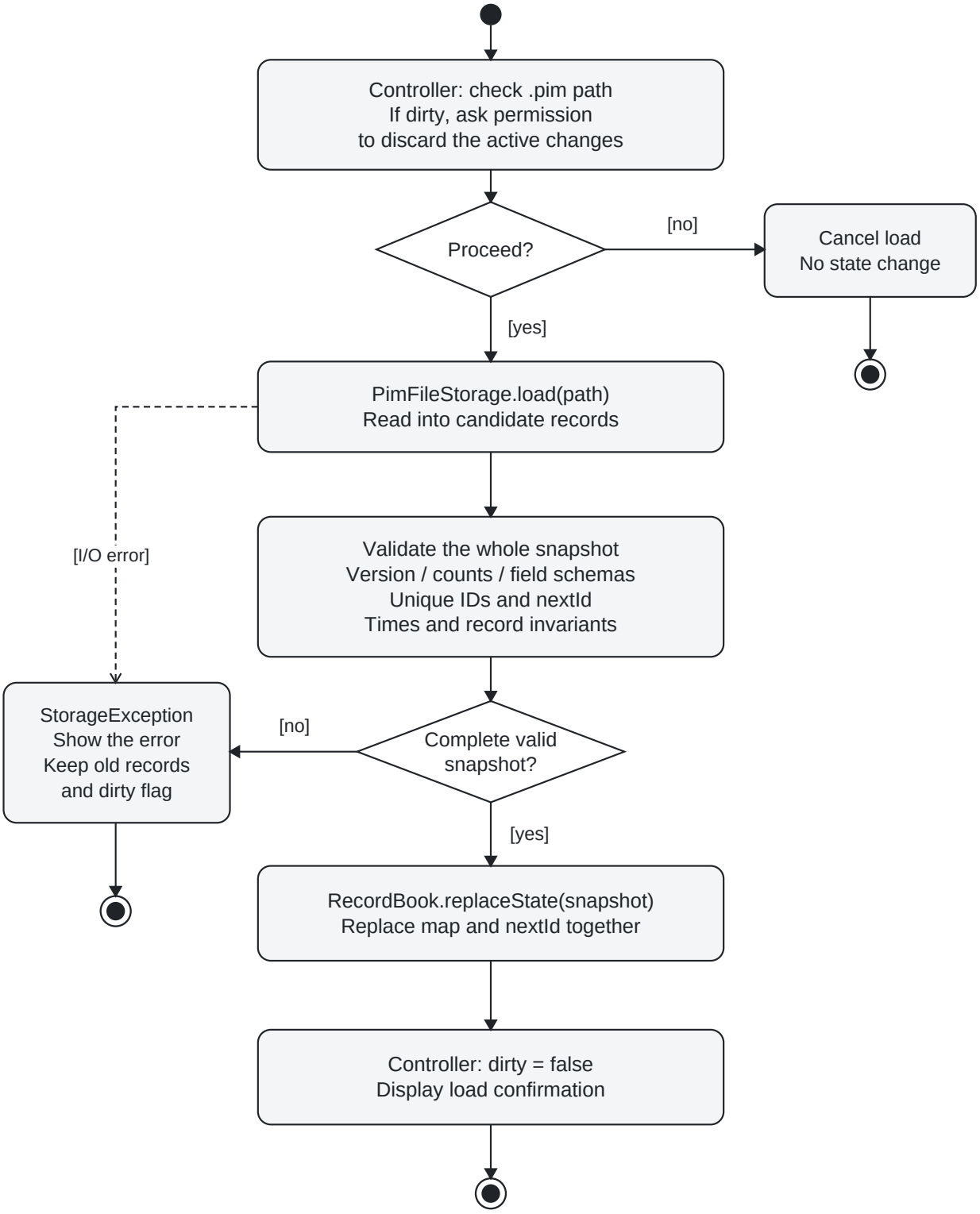


Figure 7: Load collaboration. The active book is replaced only after a complete valid snapshot is returned. Errors and cancelled confirmations retain existing records and session state.

2.5.3 Version 1 Schema

The file is UTF-8 text. It begins with exactly two header lines, followed by the declared number of record lines; there is no blank separator line. `<TAB>` below denotes an actual tab. `B64(value)` is standard Base64 of the value's UTF-8 bytes.

```
PIM<TAB>1
META<TAB>nextId<TAB>recordCount
NOTE<TAB>id<TAB>B64(text)
TASK<TAB>id<TAB>B64(description)<TAB>deadline
EVENT<TAB>id<TAB>B64(description)<TAB>start<TAB>alarm
CONTACT<TAB>id<TAB>B64(name)<TAB>B64(address)<TAB>B64(mobile)
```

Each record uses one of the four layouts. Tags are uppercase. IDs, nextId and recordCount are decimal integers, and time columns use the shared strict format. Save writes records in ascending ID order. An empty collection has recordCount 0.

Load accepts LF or CRLF line endings and rejects extra content. IDs must be unique, and nextId must be greater than the largest stored ID. Base64 preserves Unicode, delimiters and line breaks in text fields. It does not encrypt the data. The file format does not use Java object serialisation.

2.6 Controller and Console Interfaces

2.6.1 Command and CommandParser

`CommandType` contains CREATE, UPDATE, SEARCH, PRINT, DELETE, SAVE, LOAD, HELP and EXIT. `Command` holds `CommandType type` and `String body`; its constructor and getters declare no checked exceptions.

```
Command(CommandType type, String body)
CommandType getType()
String getBody()
Command parse(String line) throws CommandParseException
Map<FieldName, String> parseFields(String text)
    throws CommandParseException
```

The last two operations belong to `CommandParser`. Its no-argument constructor declares no checked exceptions. `parse` identifies the command and keeps its body. `parseFields` reads field names and quoted values, rejecting duplicate or malformed assignments. The factory checks whether each field applies to the record and whether its value is valid. Search expressions are passed unchanged to the criterion parser.

```
create <type> <field="value"> ...
update <id> <field="value"> ...
search <expression>
print <id|all>
delete <id>
save "<path.pim>"
load "<path.pim>"
help
help search
exit
```

Commands use lower-case names. The controller checks the arguments and converts IDs and paths. Invalid syntax raises `CommandParseException`. If a `print` ID has invalid syntax or exceeds `Long.MAX_VALUE`, the error gives the valid forms and a `print 1` example. Updates cannot change the ID or record type. Quoted paths and values keep their spaces. The factory parses time strings for create and update commands.

2.6.2 PimController

The controller holds final references to `RecordBook book`, `CriterionParser criterionParser`, `PimFileStorage storage`, `CommandParser commandParser` and `ConsoleView view`. Its mutable flags `running` and `dirty` both start false.

```
PimController(RecordBook book, CriterionParser criterionParser,
              PimFileStorage storage, CommandParser commandParser,
              ConsoleView view)
void run()
void execute(Command command) throws PimException
```

The constructor receives the services listed above and declares no checked exceptions. `run` displays the welcome message, then reads, parses and executes commands in a loop. It reports recoverable errors and continues with the next command. EOF cancels any pending confirmation and ends the session without automatic saving or repeated prompts. An input-stream failure also ends the session. If changes are unsaved, the controller warns the user while output is available. `execute` runs one command and converts confirmation I/O errors to `CommandParseException`.

Successful create, update and delete commands set dirty to true. An accepted update counts as an edit even when the supplied values equal the current ones. A successful save or load clears dirty, and loading restores the counter from the file. Cancelling overwrite, load or exit preserves the session and files; cancelling exit keeps the application running. The controller owns dirty state because it tracks the current session.

2.6.3 ConsoleView and Main

`ConsoleView` holds final `BufferedReader input` and `PrintStream output` references. A mutable `boolean inputClosed` records EOF or a read failure to suppress further prompts.

```
ConsoleView(BufferedReader input, PrintStream output)
Optional<String> readCommand() throws IOException
boolean confirm(String prompt) throws IOException
void showRecord(PIR record)
void showRecords(List<PIR> records)
void showMessage(String message)
void showError(String message)
void showWelcome()
void showHelp()
void showSearchHelp()
```

`readCommand` returns the next line, or an empty optional at EOF. Confirmation accepts `y` or `yes` regardless of case; other replies or EOF cancel. Display methods declare no checked exceptions.

The welcome message shows how to get help and create a note. Per FR-ERR-02, help lists commands and arguments. `help search` lists record types and searchable fields, then explains the search syntax with examples. Record output shows the ID and all applicable fields.

`Main` has no public constructor. Its `public static void main(String[] args)` method creates the services and starts the session. It declares no checked exceptions.

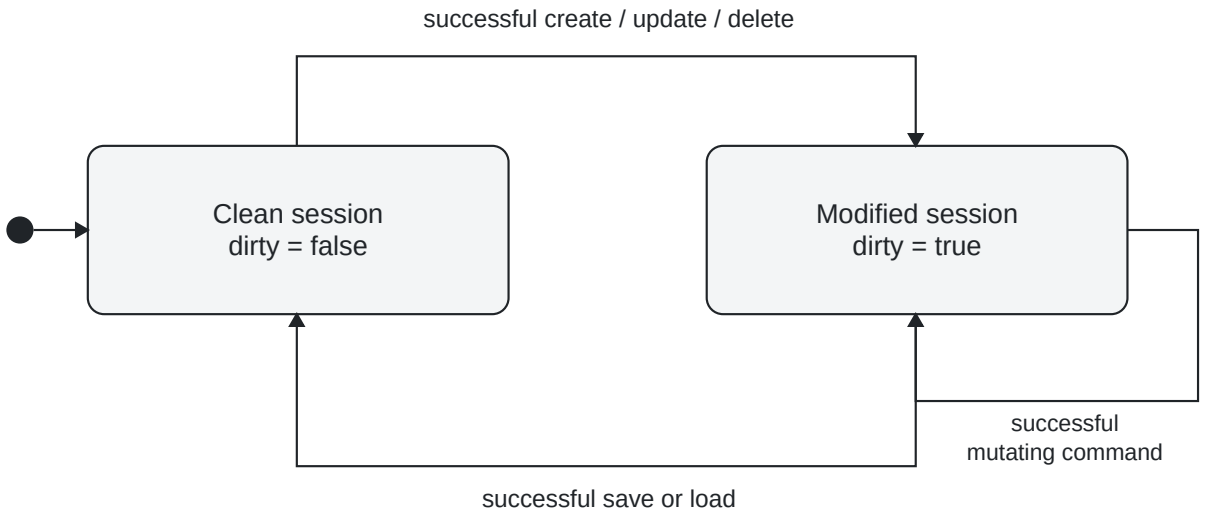


Figure 8: Controller-owned session state. Successful mutating commands mark the collection unsaved; successful save/load clears the flag. Reads, failures and cancellations preserve it.

2.7 Failure and State Contracts

Session state consists of the active records, the next ID and the controller's dirty flag. Parsing and lookup errors leave this state unchanged. The table summarises how failures and cancellations affect the session and saved files.

Condition	**Owner**	**Observable guarantee**
Rejected creation	RecordBook / factory	Keep the session unchanged. Do not consume an ID or write to disk.
Invalid update or unknown update/delete ID	RecordBook / factory	Reject the whole operation. Keep all fields, session state and files unchanged.
Rejected load, including an invalid later record	Storage / controller	Discard the candidate snapshot and keep the session unchanged. Do not modify the source file.
Save failure	Storage / controller	Keep the session unchanged. Failed temporary-file writes leave the target untouched; a failed non-atomic replacement may affect the old file.
Field absent in a query	Criterion	The comparison is false; NOT makes it true. Use a type condition to restrict the query to one record type. Search leaves the session unchanged.
Valid equal-value update	RecordBook / controller	Keep the record values and next ID, but mark the session unsaved. Do not change the saved file.
User rejects overwrite, load or exit	Controller / view	Keep the session and files unchanged. A cancelled exit leaves the application running.
End-of-input	Controller / view	Cancel pending confirmation and end the session. Do not save automatically or repeat prompts. Warn about unsaved changes while output is available.

3 Example: Search and Update an Event

3.1 User Goal and Demonstration

The user finds a consultation event, changes its start and alarm times, and prints the result. These screenshots show the application running in IntelliJ IDEA. The session starts empty, so the created event receives ID 1.

```
pim> create event description="COMP3211 project consultation" start="2026-11-16 14:00" alarm="2026-11-16 13:30"
Created record 1. Unsaved changes.
pim> search type = event && description contains "consultation" && (start < "2026-11-18 00:00" || alarm = "2026-11-16 13:30")
1 matching record:
  id: 1
  type: event
  description: COMP3211 project consultation
  start: 2026-11-16 14:00
  alarm: 2026-11-16 13:30
pim>
```

Screenshot 1. Create an event and find it using type, text and grouped time conditions.

```
pim> update 1 start="2026-11-18 16:00" alarm="2026-11-18 15:30"
Updated record 1. Unsaved changes.
pim> print 1
  id: 1
  type: event
  description: COMP3211 project consultation
  start: 2026-11-18 16:00
  alarm: 2026-11-18 15:30
```

Screenshot 2. Update both times and print the event to check the new values.

3.2 Object Collaboration

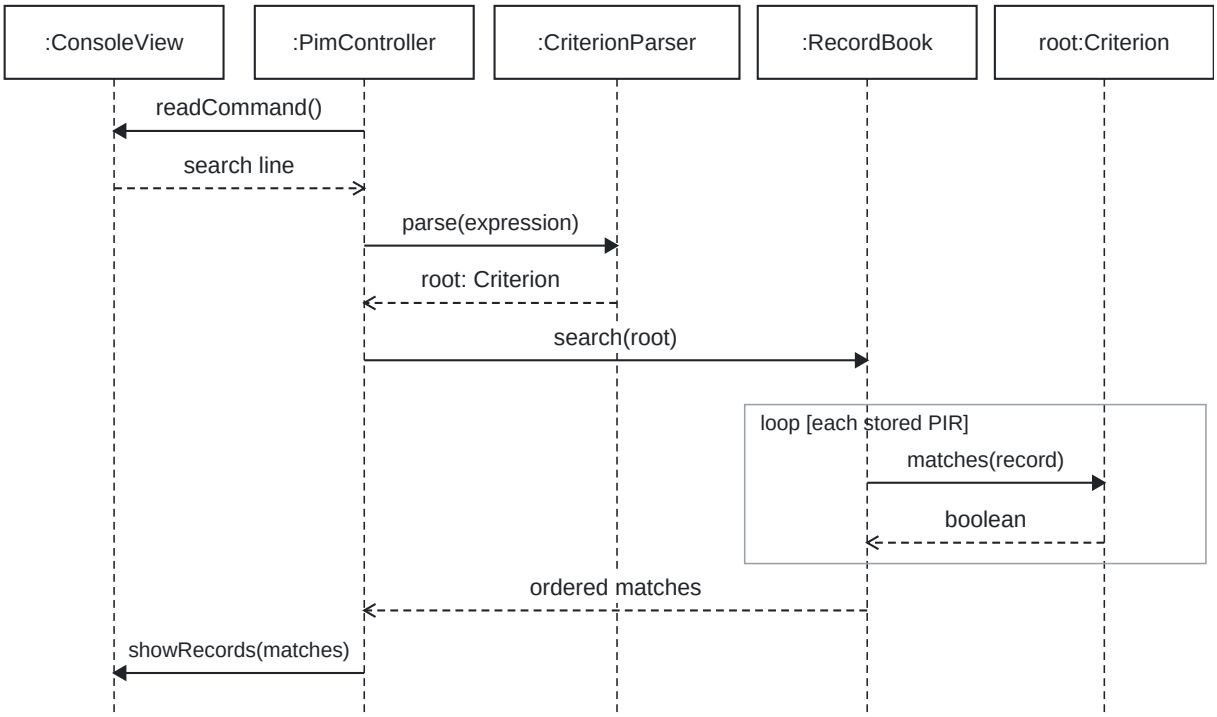
1. `PimController` reads the search command through `ConsoleView`. `CommandParser` identifies SEARCH and returns a `Command`. The controller then passes its expression to `CriterionParser`.
2. `CriterionParser.parse` builds the condition tree in Figure 5. It applies the operator precedence and parentheses before the search runs. Invalid syntax raises an error without changing records.
3. `RecordBook.search` calls `matches` on each record. The consultation event satisfies the type, description and grouped time conditions. The book sorts the matches by ID, and the view prints their IDs and fields.
4. The user takes the ID from the search result and enters an update command. The controller parses that ID and the new field values, then calls `book.update(id, changes)`. Using the ID selects the intended record even when several records have the same description.
5. The book retrieves the original event and calls `RecordFactory.updated`. The factory combines the supplied fields with the unchanged values and constructs a new event. Its constructor checks the complete result, including whether the alarm is at or before the start.
6. Only a valid replacement is written to the map. If validation throws an exception, execution stops before that write, and the original record stays intact. An update containing both a valid description and an invalid alarm therefore changes neither field.
7. After the book returns successfully, the controller marks the session dirty and asks the view to display confirmation. PRINT retrieves the replacement. Repeating the original search excludes it because neither of the original time conditions is now true.

The order is visible in `RecordBook.update`:

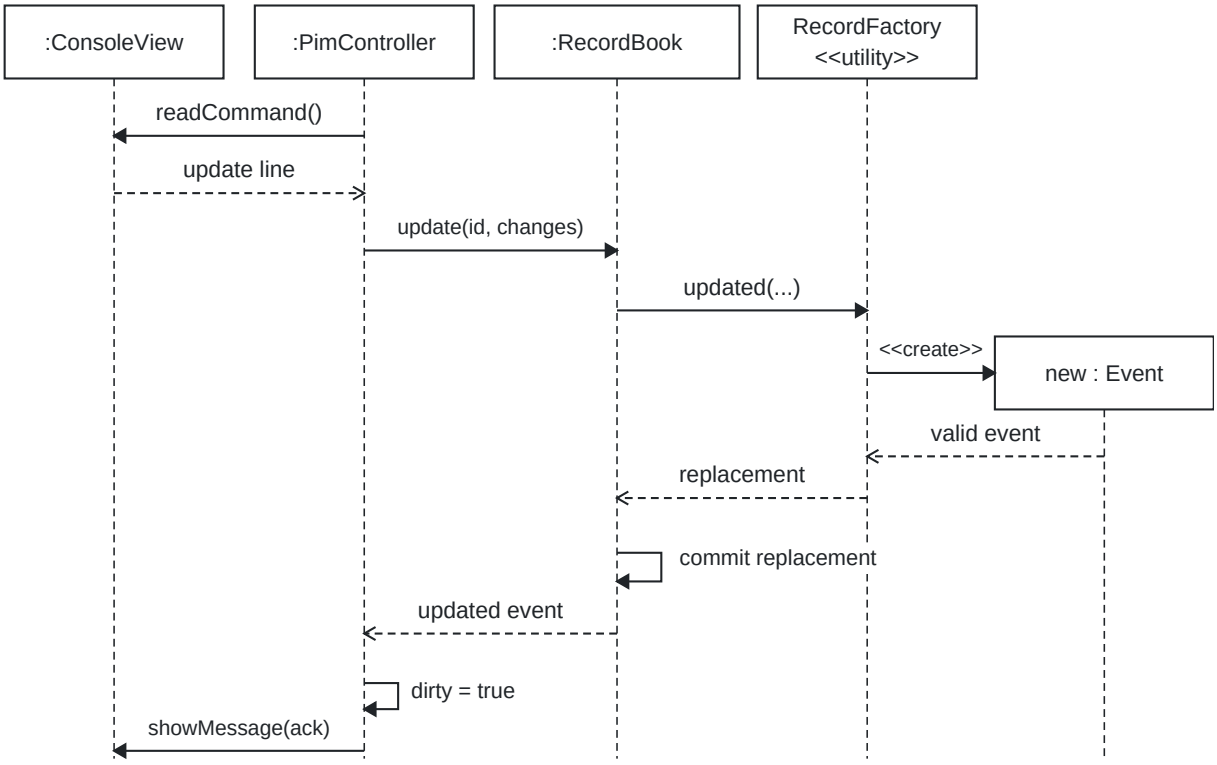
```
PIR replacement = RecordFactory.updated(get(id), changes);
recordsById.put(id, replacement);
```

The first line constructs and validates the replacement. The second line stores it. The original immutable record is not edited during validation. This is why the rejected update in Section 3.3 leaves all fields unchanged.

A. Search: construct a criterion, evaluate records, display matches



B. Update: validate both time fields before committing a replacement



Solid arrows: calls. Dashed arrows: returned values.
CommandParser envelope and field parsing are omitted; see Section 2.6.

Figure 9: Search/update collaboration. Solid arrows are calls and dashed arrows are returns. The static factory validates the replacement before the book commits it and the controller marks the session dirty.

3.3 Rejected Update and Preserved State

The next command changes the description and sets the alarm to `2026-11-18 17:00`, after the event's 16:00 start. PIM rejects the whole update. Printing the event again shows that its description and both times are unchanged. The rejected command changes neither field.

```
pim> update 1 start="2026-11-18 16:00" alarm="2026-11-18 15:30"
Updated record 1. Unsaved changes.
pim> print 1
  id: 1
  type: event
  description: COMP3211 project consultation
  start: 2026-11-18 16:00
  alarm: 2026-11-18 15:30
pim> update 1 description="Should not be saved" alarm="2026-11-18 17:00"
Error: The alarm must be at or before the event start.
pim> print 1
  id: 1
  type: event
  description: COMP3211 project consultation
  start: 2026-11-18 16:00
  alarm: 2026-11-18 15:30
pim>
```

Screenshot 3. An invalid alarm rejects both requested changes; the printed record retains the previous values.

3.4 Save and Load Round Trip

The user saves the collection, changes the description to `Temporary edit`, and prints the record. Loading the saved file asks whether to discard the unsaved change. After confirmation, the original description is restored and the updated start and alarm times remain unchanged.

```
pim> save "consultation-demo.pim"
Saved to consultation-demo.pim.
pim> update 1 description="Temporary edit"
Updated record 1. Unsaved changes.
pim> print 1
  id: 1
  type: event
  description: Temporary edit
  start: 2026-11-18 16:00
  alarm: 2026-11-18 15:30
pim> load "consultation-demo.pim"
Discard unsaved changes and load? [y/yes] y
Loaded from consultation-demo.pim.
pim> print 1
  id: 1
  type: event
  description: COMP3211 project consultation
  start: 2026-11-18 16:00
  alarm: 2026-11-18 15:30
pim>
```

Screenshot 4. Save the event, make a temporary edit, then load the file and check that the saved values return.

Figures 6 and 7 show the save and load calls.

`ConsoleTests.eventScenario` checks this workflow and verifies that loading also restores the next record ID.

4 Design Evaluation

4.1 Efficiency and Resource Use

Let n be the number of records, m the number of nodes in a search criterion, k the number of matches, and B the encoded file size in bytes. The hash map gives expected $O(1)$ access for record lookup, deletion and replacement. Validation and construction of a replacement add their own cost. `getAll` sorts by ID, taking $O(n \log n)$ time and $O(n)$ space for the returned list.

Search performs at most $O(nm)$ condition checks, excluding substring comparison costs. Sorting the matches adds $O(k \log k)$. Short-circuit AND and OR can skip some checks.

Creating and validating a snapshot takes $O(n)$. Sorting its records for saving adds $O(n \log n)$, and encoding and writing take $O(B)$. The overall save cost is therefore $O(n \log n + B)$.

Loading takes $O(B)$ for decoding and expected $O(n)$ for duplicate-ID checks and map construction. It keeps the candidate collection in memory until validation finishes, so extra memory grows with the decoded data. Commands run sequentially; concurrent access is outside scope.

4.2 Modularity, Extensibility and Trade-offs

D1–D4 in Section 1.4 give the reasons and costs of the main abstractions. Record validation and collection operations are available through model APIs. Confirmation and EOF handling belong to the controller and view, where terminal input is available.

A new record type requires a permitted PIR subclass, enum and factory updates, field access, output formatting, and a file-format compatibility decision. A new predicate requires a criterion implementation and a parser rule; existing Boolean combinators remain reusable.

We chose a linear scan to keep updates simple for a small local collection. The controller asks the view to display results after each command. Tab-separated fields and Base64 keep text boundaries intact using standard-library encoding. The trade-off is that encoded text is harder to read manually, and loading needs space for a complete candidate collection. File-operation failures preserve the in-memory session; the persistence section explains the disk behaviour.

4.3 User-story-to-design Mapping

Story	Operation / component	Design commitment
US1	`RecordBook.create`, `RecordFactory.create`	Validate the concrete record before allocating its stable ID.
US2	Factory creation of `Note`	Preserve validated plain text.
US3	Factory creation of `Task`	Validate description and deadline.
US4	Factory creation of `Event`	Validate description, start and stored alarm; alarm no later than start.
US5	Factory creation of `Contact`	Preserve name/address and string-valued mobile number.
US6	`RecordBook.update`, `RecordFactory.updated`	Validate before replacement. Reject invalid edits completely; accepted updates mark the session unsaved.
US7	`CriterionParser.parse`, `Criterion.matches`, `RecordBook.search`	Apply grammar, precedence and typed comparisons. A comparison against a missing field is false.
US8	`RecordBook.get/getAll`; `ConsoleView.showRecord/showRecords`	Print complete fields and stable IDs; lists ordered by ID.
US9	`RecordBook.delete`	Remove the selected ID only; retain allocation history.
US10	`RecordBook.snapshot`, `PimFileStorage.save`	Write the versioned `.pim` collection. Confirm overwrite; preserve the session if saving fails or is cancelled.
US11	`PimFileStorage.load`, `RecordBook.replaceState`	Validate fully before restoring records and nextId. Failed or cancelled loads preserve the session.

The requirements coverage report will map the user stories and the SRS functional and non-functional requirements to implementation and verification evidence.

4.4 Implementation Checks

The supplied test script passes all 28 model tests and seven console tests. Model tests cover record validation, search and persistence. Console tests check the command loop, confirmations and recovery from rejected commands. `ConsoleTests.eventScenario` joins search, update, save and load in one session. The example in Section 3 was also run through `app.Main`, including loading the saved data in a new process.

The Requirements Coverage Report will list the NFR-04 benchmark results and instructions for repeating the measurement. The Model Line Coverage report will list the coverage totals and uncovered cases.

4.5 Functional Test Cases and Expected Outcomes

The tables below link successful operations and failure cases to their JUnit tests.

4.5.1 Test setup and successful operations

The model tests call record, criterion and storage operations directly. Console tests supply command lines through a `StringReader` and capture terminal output in memory. This lets them check error messages, confirmation prompts and the next command in the same session.

Persistence tests use temporary files and directories. Their assertions compare field values and the next ID after reloading. Tests that reject an edit also compare the original record or collection after the error.

Function	Input and expected result	JUnit methods
Create	Create a note, task, event and contact with all required fields. The four stored records have distinct IDs and retain the supplied text and time values.	`RecordTests.createRecords`
Update	Move an event's start and alarm to valid times in one update. Both times change together. Its ID and description are preserved, and the original immutable object retains its old values.	`RecordTests.updateEvent`
Search	Evaluate the grouped consultation-event query before and after moving the event. It matches the original event and excludes the revised one. Search results are returned in ID order.	`CriteriaTests.eventSearch` `RecordTests.orderedResults`
Save	Save a snapshot containing all four types, Unicode text and `nextId=20`. The file has the expected header and metadata, and its record lines appear in ascending ID order.	`StorageTests.roundTrip`
Load	Load the saved snapshot. Every field and ID is restored, and the next creation receives ID 20. Loading an empty collection also succeeds.	`StorageTests.roundTrip` `StorageTests.supportedLayouts`

`ConsoleTests.eventScenario` joins search, update, save and load through the command loop. It checks the event's updated fields after reloading and confirms that new records use the restored ID counter.

4.5.2 Invalid input, file failures and preserved state

Normal creation assigns IDs inside `RecordBook`. Duplicate-ID inputs therefore enter through snapshot construction or file loading. The checks below cover both rejection and the resulting state.

Function	Failure case and expected result	JUnit methods
Create	Supply a task deadline of `2026-02-29 09:00`. The invalid date is rejected, the collection remains empty and the next valid creation receives ID 1. A separate parsing check rejects the invalid clock time `2026-02-01 24:00`.	`RecordTests.invalidCreation` `RecordTests.strictTimes`
Update	Change an event's description and move its start before its existing alarm. Validation rejects the complete update. The original record and the next ID are preserved.	`RecordTests.invalidUpdate`
Search	Use an invalid date, unsupported operator or incomplete expression. Parsing raises `CriterionParseException` with a position and reason. For an invalid date, it retains the validation error as its cause.	`CriteriaTests.invalidQueries`
Load	Repeat ID 2 in a file or repeat a record in a snapshot. File loading raises `StorageException` at the duplicate record. Snapshot construction raises `ValidationException`.	`StorageTests.badSnapshotLines` `RecordTests.invalidSnapshots`
Load	Place a malformed event after a valid note in the file. The load reports an error on line 4. The active note, next ID 2 and unsaved state are preserved.	`ConsoleTests.failedLoad`
Load	Insert an invalid UTF-8 byte into a file. Loading raises `StorageException` with a UTF-8 diagnostic.	`StorageTests.malformedFileEncoding`
Save	Use a missing parent directory or an existing directory as the target. Saving raises `StorageException`. Directory contents are preserved. The console check also verifies that the current records and unsaved state survive the failure.	`StorageTests.failedSave` `ConsoleTests.failedSave`

`ConsoleTests.cancelConfirmations` separately verifies that declining overwrite, load or exit preserves the current data. File failure checks use deterministic invalid destinations. Unusual file-system failures during atomic moves or cleanup are not covered by these tests; the persistence section describes how the code handles them.